

ETRI 2026 E2E Driving — 최종 제출물 (EXT-FULL-v7)

팀 쉬엠팀의 리더보드 최종 결과 **L2_avg 0.117067** 모델 제출물입니다. 이 문서는 제출물 구성, 실행 환경, 추론·검증·학습 방법을 순서대로 설명합니다. 모든 명령은 이 폴더(`etri_e2e_final/`)에서 실행합니다.

L2_1s	L2_2s	L2_3s	L2_avg	__flops__	RTX 4090 1회 forward
0.065436	0.115299	0.170465	0.117067	743,777,233,600	약 31 ms

모델은 6개 카메라 영상에서 궤적 후보 15개를 만드는 end-to-end 네트워크입니다. 후보 가운데 5초 끝점이 제공된 목표 점에 가장 가까운 것을 좌표 변경 없이 출력합니다. 구조와 학습 전략은 `docs/TECHNICAL_REPORT.md`에서 설명합니다.

1. 제출 요구 항목과 위치

제출 요구 항목	위치	설명
Docker 이미지	<code>../docker_image/etri-e2e-ext_final.tar.gz</code>	<code>docker load</code> 로 불러오는 이미지 파일입니다(이미지명 <code>etri-e2e-ext:final</code>). <code>docker/Dockerfile</code> 로 같은 이미지를 다시 빌드할 수도 있습니다
리더보드 성능 checkpoint	<code>checkpoints/ext_full_v7.pth</code>	리더보드 0.117067 제출에 사용한 checkpoint입니다. 학습 단계별 시작 checkpoint도 같은 폴더에 있습니다
학습·추론 코드 (모델 포함)	<code>src/</code>	리더보드 제출 때 사용한 코드를 수정 없이 담았습니다
데이터셋 준비 및 코드 작동 방법	이 문서 3~5절, <code>docs/DATA_PREP.md</code>	테스트·학습 데이터 준비와 추론·학습 실행 방법입니다
사용 기술 및 전략 설명서	<code>docs/TECHNICAL_REPORT.md</code>	입력 사용 방식, 모델 구조(그림 포함), 학습 방법, 검증 전략, 실험 결과입니다
참고 자료	<code>docs/COMPLIANCE.md</code> , <code>docs/VALIDATION.md</code>	대회 규정 항목별 대응과 근거, 제출물 검증·재현 결과입니다

모든 문서는 Markdown(`.md`)과 PDF(`.pdf`)로 함께 제공합니다. 내용은 같습니다.

2. 실행 환경

항목	요구 사항
GPU	추론·검증: NVIDIA GPU 1장(RTX 4090에서 확인). 학습: 6절 참고
드라이버	CUDA 12.8을 지원하는 NVIDIA 드라이버(570 이상, 595.91에서 확인)
소프트웨어	Docker, NVIDIA Container Toolkit(<code>docker run --gpus</code> 가 동작해야 합니다). 인터넷 연결은 필요 없습니다
디스크	이미지 약 13 GB(압축 파일 4.4 GB), 테스트 데이터 약 68 GB, 학습할 경우 원본 약 199 GB와 준비 결과 약 70 GB

제출 zip의 구성은 다음과 같습니다.

쉬웠음청년_20260923_0.117067/	
└─ docker_image/	
└─ etri-e2e-ext_final.tar.gz	Docker 이미지 (docker save 결과)
└─ etri-e2e-ext_final.tar.gz.sha256	파일 확인용 해시
└─ etri_e2e_final/	이 폴더
└─ README.md / README.pdf	이 문서
└─ docs/	설명서 (.md, .pdf), 아키텍처 그림
└─ src/	모델, 학습, 추론, 데이터 준비 코드
└─ checkpoints/	제출 모델과 학습 단계별 checkpoint (7개)
└─ data/	학습용 파생 캐시, 실행 메타데이터, 검증용 학습 clip 8개
└─ docker/	Dockerfile
└─ scripts/	실행 스크립트 (아래 표)
└─ tools/	컨테이너 경로 구성, 학습 영상 준비, 검증 도구
└─ validation/	검증용 테스트 clip 20개, 참조 예측, 검증 결과
└─ layout.json	src/·checkpoints/·data/ 파일과 코드 내부 경로의 대응표

스크립트	용도
scripts/build_image.sh	Dockerfile로 이미지를 빌드합니다(이미지 파일을 쓰면 필요 없습니다)
scripts/validate.sh	동봉 clip으로 정합성, FLOPs, GPU 추론 시간을 확인합니다
scripts/infer.sh	테스트 1,125 clip을 추론하고 리더보드 제출 파일을 만듭니다
scripts/prepare_train.sh	원본 학습 데이터를 학습 코드가 읽는 형태로 변환합니다
scripts/train.sh	학습 1·2·3단계를 실행합니다

3. 빠른 시작: 이미지 불러오기와 검증

cd 쉬웠음청년_20260923_0.117067/docker_image	
sha256sum -c etri-e2e-ext_final.tar.gz.sha256	# "OK"가 나오면 파일이 온전합니다
docker load -i etri-e2e-ext_final.tar.gz	# "Loaded image: etri-e2e-ext:final" (수 분)
cd ../etri_e2e_final	
bash scripts/validate.sh	# RTX 4090 약 20초, 결과: validation/result/valid
ation.json	

validate.sh는 동봉한 clip 28개로 다음 항목을 확인합니다. RTX 4090 기준 기대값은 다음과 같습니다.

validation.json 항목	의미	기대값
flops	1회 forward 전체 FLOPs	743,777,233,600 (제출값과 같아야 합니다)
latency_ms.median	1회 forward 시간(전처리 제외)	약 31 ms
fixtures_vs_B200.same_selected_candidate	학습 clip 8개, 학습 서버 예측과 같은 후보를 고른 수	8/8
test_vs_submitted.same_selected_candidate	테스트 clip 20개, 실제 제출 파일과 같은 후보를 고른 수	18/20 이상
test_vs_submitted.max_abs_m_median	테스트 clip 좌표 차이 중앙값	약 0.0015 m

GPU 종류가 다르면 BF16 연산 차이로 좌표가 mm 단위로 달라집니다. 5초 끝점과 목표점의 거리가 거의 같은 후보가 2개 있는 clip에서는 선택 후보가 바뀔 수 있습니다(docs/VALIDATION.md 1절).

4. 추론: 테스트 데이터 → 리더보드 제출 파일

```
bash scripts/infer.sh /path/to/test OUT_DIR
```

- **입력:** 대회에서 배포한 테스트 폴더를 그대로 지정합니다. 1,125개 clip 폴더 (<clip_hash>/camera_*/frame_{0..-30}.jpg, ego_pose.parquet, calibration.parquet)가 모두 있어야 합니다. 전처리(왜곡 보정, crop, resize, ego 상태 계산, 목표점 읽기)는 추론 코드가 수행하므로 별도로 준비할 것이 없습니다.
- **출력:** OUT_DIR/package/submission.zip이 리더보드에 올리는 파일입니다. OUT_DIR/work/predictions.json은 clip별 6점 좌표, OUT_DIR/work/flops_report.json은 FLOPs 기록입니다.
- **소요 시간:** RTX 4090에서 약 7분입니다.
- **결과:** RTX 4090에서 1,118/1,125 clip이 실제 제출 파일과 같은 후보를 고르고, 전체 가중 좌표 차이 평균은 0.9 mm입니다. 학습 서버(B200)에서는 1,125 clip 모두 제출 파일과 완전히 같습니다.

5. 학습 데이터 준비

```
bash scripts/prepare_train.sh /path/to/train PREP # 1회, 결과 약 70 GB
```

- **입력:** 대회 제공 train 376 scenario입니다. 배포 형식인 <scenario>.tar 376개가 있는 폴더를 지정하면 됩니다. 압축을 푼 <scenario>/ 폴더도 받습니다.
- **출력:** PREP/etri_768/ (왜곡 보정한 768×432 영상 676,800장)과 PREP/meta_train/ (원본 parquet)입니다.
- **소요 시간:** 병렬 작업 수는 JOBS 환경변수로 정하고 기본값은 8입니다. 24개 병렬에서 약 12분이 걸렸습니다.
- **준비 결과는 학습에 사용한 캐시와 바이트 단위로 같습니다(376 scenario 전체 확인).** 자세한 내용은 docs/DATA_PREP.md에 있습니다.

6. 학습 (선택)

```
bash scripts/train.sh 1 PREP OUT_DIR # 1단계
bash scripts/train.sh 2 PREP OUT_DIR # 2단계
bash scripts/train.sh 3 PREP OUT_DIR # 3단계 (제출 모델 레시피)
CKPT=OUT_DIR/stage3/ckpt_step6344.pth bash scripts/infer.sh /path/to/test OUT_INFER # 직접 학습한 모델로 추론
```

단계	시작점	학습 대상	update	시간 (B200 1장)	결과 위치	제출물의 해당 checkpoint
1	stage1_initializer.pth (공개 백본 + 무작위 초기화)	전체 모델	24,931	약 3.6시간	OUT_DIR/stage1/A2-H4-PROGRESS-FULL-s1/	stage1_h4_progress_full.pth
2	동봉한 1단계 checkpoint	전체 모델, 낮은 학습률	38,070	약 5.6시간	OUT_DIR/stage2/L-FULL6-s1/	stage2_l_full6.pth
3	동봉한 2단계 checkpoint	planner (몸통 동결), 후보 15개	6,344	약 0.7시간	OUT_DIR/stage3/ckpt_step6344.pth	ext_full_v7.pth (제출 모델)

- **단계 연결:** 각 단계는 제출물에 동봉한 이전 단계 checkpoint에서 시작합니다. 2단계 코드는 시작 checkpoint의 sha256을 확인하므로, 단계를 따로 실행해도 제출 모델과 같은 경로를 재현합니다.
- **GPU 조건:** 모든 단계를 NVIDIA B200(180 GB) 1장으로 학습했습니다.

- 1·2단계 코드는 시작할 때 GPU 여유 메모리가 약 174 GiB(170,000 + 8,192 MiB) 이상인지 검사합니다. 그래서 비어 있는 B200급 GPU가 필요합니다. 실제 최대 사용량은 약 37 GB입니다.
- 3단계에는 이 검사가 없고, 실제 사용량은 약 15 GB입니다(RTX 4090에서 실행 가능).
- 24 GB GPU에서 1·2단계의 동작만 확인하려면 docs/VALIDATION.md 3절의 tools/small_gpu 사용법을 참고합니다. 이 방법은 microbatch만 줄이고 계산식은 바꾸지 않습니다.
- **재현 정도:** 같은 seed로 다시 학습하면 tune 평가가 1단계 0.106857(원래 0.106606), 3단계 0.087666(원래 0.087320)입니다. BF16과 비결정적 CUDA 연산 때문에 비트 단위로 같지는 않습니다.

7. 자주 겪는 문제

증상	해결
GPU가 여러 장이어서 특정 GPU를 쓰고 싶습니다	<code>GPU=3 bash scripts/infer.sh ...</code> 처럼 번호를 지정합니다. 지정한 GPU만 컨테이너에 보이고, 컨테이너 안에서는 장치 0이 됩니다
출력 폴더의 파일을 지울 수 없습니다	컨테이너가 root 권한으로 파일을 만듭니다. <code>sudo rm -rf OUT_DIR</code> 또는 <code>sudo chown -R \$USER OUT_DIR</code> 을 사용합니다
<code>infer.sh</code> 가 <code>assert len(clips) == 1125</code> 에서 멈춥니다	추론 코드는 테스트 1,125 clip이 모두 있는지 확인합니다. 테스트 폴더 전체를 지정합니다
1·2단계가 <code>Insufficient CUDA headroom</code> 으로 멈춥니다	GPU 여유 메모리가 약 174 GiB보다 적은 경우입니다. 비어 있는 B200급 GPU를 <code>GPU=</code> 로 지정합니다
<code>docker: could not select device driver "" with capabilities: [[gpu]]</code>	NVIDIA Container Toolkit이 설치되지 않은 경우입니다
학습이 시작 직후 <code>assert not ... exists()</code> 또는 <code>Refuse overwrite</code> 로 멈춥니다	학습 코드는 기존 결과를 덮어쓰지 않습니다. 새 <code>OUT_DIR</code> 을 지정합니다

8. 동작 방식: 컨테이너 경로 구성

학습 코드는 학습 당시의 저장소 경로(`/NHNHOME/data/sukim/adcl/...`)와 소스 파일 sha256을 검사합니다. 그래서 컨테이너가 시작할 때마다 `tools/assemble.py`가 `layout.json`에 따라 원래 경로 구조를 컨테이너 내부에 다시 만듭니다. 소스 파일은 수정 없이 복사하고, checkpoint와 데이터는 링크로 연결합니다. 제출물 폴더는 읽기 전용으로 마운트되므로 실행해도 바뀌지 않습니다. 결과는 `validate.sh`가 기본으로 쓰는 `validation/result/`와, 각 스크립트에 지정한 출력 폴더에만 생깁니다.

9. 외부 데이터

외부 데이터는 1단계 초기값에 들어간 공개 백본 가중치 하나입니다. `mmdetection3d의 cascade_mask_rcnn_r50_fpn_coco-20e_20e_nuim_20201009_124951-40963960.pth` (COCO, nuImages 학습)이고, 파일은 `checkpoints/backbone_nuimages_cascade_r50.pth`입니다. 학습 데이터는 대회 제공 train 376 scenario만 사용했습니다.